

Can Developers Rely on LLM-based Tools to Support Software Development Activities?

An Experimental Comparison of ChatGPT, Claude, and Gemini on Code Explanation and Bug Finding

Daniel Pereira — 2021237092
Eduardo Marques — 2022231584
João Cardoso — 2022222301

May 2026

Abstract

The rapid adoption of Large Language Model (LLM) based assistants in software engineering raises a practical question for technology decision-makers: can these tools be relied upon for coding-related tasks? This report presents the design, execution, and analysis of an experiment whose purpose was to compare three widely used commercial LLMs — **ChatGPT**, **Claude**, and **Gemini** — in two fundamental coding tasks: *code explanation* and *code review for bug discovery*. For code explanation, ten short C/Java snippets were graded with a rubric and analysed with ANOVA followed by directional one-tailed Student's *t*-tests at $\alpha = 0.01$. Because the first round could not separate ChatGPT from Claude, the sample size was doubled and the rubric tightened in a second round, after which ChatGPT was found to be significantly better. For bug finding, 35 buggy modules per model were annotated against a known ground truth, and *precision* and *recall* were computed. A two-way ANOVA confirmed model as a significant factor; one-sample *z*-tests against a minimum acceptable recall of 75% eliminated ChatGPT; a final two-proportion *z*-test showed that Claude significantly outperforms Gemini. The aggregated evidence supports adopting different tools for different tasks, with **Claude** recommended for code review/bug discovery and **ChatGPT** recommended for code explanation.

Disclaimer

This document was reviewed and refined with the assistance of ChatGPT and similar tools, which helped check grammar, correct typos, and enhance clarity. The charts presented in this report were generated with the assistance of Claude. All ideas, experimental design, data collection, and analysis were carried out exclusively by the authors. The overall content and conclusions remain solely the responsibility of the authors.

Contents

1	Introduction	2
2	Background	2
2.1	Code Explanation	2
2.2	Bug Finding	2
3	Problem Statement and Variables	3
3.1	Refined research question	3
3.2	Variables	3
4	Experimental Setup and Protocol	3
4.1	Tools under study	3
4.2	Evaluation Protocols	4
4.2.1	Code explanation	4
4.2.2	Bug finding	4
5	Results	5
5.1	Code explanation	5
5.1.1	Round 1 (n = 10 per model)	5
5.1.2	Round 2 (n = 20, ChatGPT vs. Claude)	6
5.2	Bug finding	7
5.2.1	Step 1: two-way ANOVA on recall	8
5.2.2	Step 2: which models meet the 75% recall floor?	9
5.2.3	Step 3: head-to-head between the survivors	10
6	Discussion	10
6.1	Two tasks, two winners	10
6.2	Methodological reflections	11
6.3	Limitations and Future Work	11
7	Conclusions and Recommendation to the CTO	11
8	Contributions of the Group Members	12
9	References	12
10	Statement on the use of AI tools	13

1 Introduction

LLM-based coding assistants have moved from novelty to everyday tool in only a few years, and most software companies are now under pressure to standardise on one of them. The promise is well known: a measurable productivity gain across virtually every coding-related activity. The risk is equally well known: the code, explanations, and reviews produced by these tools are not always correct, and the resulting reliability cost is often paid silently, downstream, after the suggestions have already been merged.

This work answers, for a specific organisational context, the question posed by the assignment:

Can developers rely on LLM-based tools to support software development activities?

To make this question tractable, the scenario suggested in the assignment statement is adopted: the CTO of an international software house has requested an evaluation of competing LLM-based tools and a recommendation for company-wide adoption. The study is restricted to coding-related tasks (excluding requirements and design activities) and, within those, to two of the four primitive tasks identified in the assignment:

- **Code explanation** — given a piece of source code, the tool must produce a natural-language description of what the code does. Quality is judged with a rubric.
- **Code review for bug discovery** — given a piece of source code containing a known set of defects, the tool must identify them. Quality is judged by *precision* and *recall* against ground truth.

The structure of the report follows the eight-step skeleton recommended in the assignment statement: problem statement, variables, hypotheses, setup, protocol, execution, analysis, and conclusions.

2 Background

LLM-based coding assistants such as ChatGPT, Claude, and Gemini have seen rapid adoption in software development workflows. Their capabilities span a range of tasks such as code generation, explanation, review, and testing. The reliability varies significantly depending on the task and the tool.

2.1 Code Explanation

Natural-language explanation of source code is a fundamental task in software development, supporting activities such as onboarding, documentation, and code reading. The quality of an explanation depends on its correctness, completeness, and clarity. In this study, each model was given the same code snippets and prompted to produce a natural-language description, which was then graded by human evaluators against a common set of criteria.

2.2 Bug Finding

For code review and bug finding, the cost asymmetry between missed defects and false alarms is well established in the software testing literature: a false positive costs a developer a few minutes of investigation, while a missed bug can reach production and cause significantly greater damage. This asymmetry justifies prioritising recall over precision as the primary evaluation criterion, and motivates setting a minimum acceptable recall threshold of 75%.

3 Problem Statement and Variables

3.1 Refined research question

The general assignment question is refined into two task-specific questions:

- **RQ1 (code explanation).** Among ChatGPT, Claude, and Gemini, which model produces better natural-language explanations of source code, as measured by a rubric-based grade?
- **RQ2 (bug finding).** Among ChatGPT, Claude, and Gemini, which model is more reliable at identifying bugs in code, as measured by recall against a known ground truth?

3.2 Variables

RQ1 — Code explanation.

- **Dependent variable:** rubric grade in $[0, 20]$ assigned to each code-explanation response (computed as the average of the three evaluators scores for that response).
- **Independent variable:** the LLM-based tool (ChatGPT, Claude, Gemini).

RQ2 — Bug finding.

- **Dependent variables:** per-case precision and recall, computed against a manually curated ground truth (derived from the per-case counts of true positives, false positives, and false negatives).
- **Independent variables:** the LLM-based tool (ChatGPT, Claude, Gemini), the programming language (C, Java), and the difficulty level (Easy, Medium, Hard).

Controlled variables (both RQs). Same prompt template per task; same code snippets across models; same human graders; same ground-truth bug list; same model usage mode (chat web interface, default configuration, no extended thinking or web search enabled, single-turn).

Nuisance variables identified and mitigated.

- Grader bias — mitigated by pre-defining the rubric before any grading took place; in the second round of code explanation the rubric was tightened to reduce ceiling effects.
- Order/learning effects — mitigated by submitting the same code to each model in independent sessions on the same day, with cleared context.

4 Experimental Setup and Protocol

4.1 Tools under study

Three commercial, general-purpose LLM-based assistants were selected:

- **GPT-5.5 Instant** (OpenAI).
- **Claude Sonnet 4.6** (Anthropic).
- **Gemini 3** (Google).

All three were used, through the standard web chat interface, with their default configuration: no system prompt customisation, no web browsing enabled, no code execution, and a fresh chat per task (cleared context). This corresponds to the most realistic “out-of-the-box” developer experience.

4.2 Evaluation Protocols

4.2.1 Code explanation

A pool of short source-code snippets (Java, 15–80 lines each) was selected from a teaching repository and lightly adapted. Each snippet was given to each model with the same fixed prompt:

“You are a code explanation assistant. Analyze the Java code provided below and explain it clearly and concisely. Your explanation must follow these rules: Maximum 10 lines of continuous text — do not exceed this limit. Cover the following aspects: what the code does overall (final goal / purpose), the key logic or algorithm used (how it works), the main Java concepts or features involved (e.g., recursion, generics, threads), the time or space complexity if relevant, and any notable design decision or potential limitation. Write your explanation in Portuguese, formatted as a PDF document. Be direct and precise.”

Each response was graded independently by the three evaluators with a rubric in [0,20] covering:

- **Correctness** — whether the explanation is accurate (40%).
- **Completeness** — whether the main aspects are covered (20%).
- **Clarity** — whether the explanation is easy to understand (30%).
- **Overall goal identification** — whether the code’s overall goal is identified (10%).

The final score for each response was computed as the average of the grades assigned by the three evaluators, and standard deviations were also calculated to assess the level of agreement between them. Both the averages and standard deviations were computed using a Python script.

The first round used $n = 10$ explanations per model. As shown in Section 5, this round proved unable to distinguish ChatGPT from Claude, so a second round was executed with $n = 20$ explanations per model (ChatGPT and Claude only) and a tightened rubric in which partial credit was made stricter to avoid ceiling effects.

4.2.2 Bug finding

A set of 35 buggy code modules (20 in C, 15 in Java) was prepared, stratified by difficulty into *Easy* ($n = 8$), *Medium* ($n = 16$), *Hard* ($n = 11$). Difficulty was defined by the type, subtlety, and number of the planted defects.

For each module, a ground-truth list of bugs was produced and verified by two evaluators before any model was queried. Each module was then submitted independently to each of the three models with the prompt:

“Analyze the following code and identify all bugs present. Generate a PDF file where the first line states the total number of bugs found in the format “Bugs found: N”, followed by a numbered list of all bugs, one per line, each with a clear and concise description. Do not include any introduction, additional explanations, fix suggestions, headers, footers, or any other content outside this format.”

For each (model, module) pair the response was scored by counting:

- **TP** (true positives) — reported bugs that match the ground truth.
- **FP** (false positives) — reported bugs that are not in the ground truth and are not actual defects.

- **FN** (false negatives) — ground-truth bugs that were not reported.

Then Precision = TP/(TP + FP) and Recall = TP/(TP + FN).

5 Results

5.1 Code explanation

5.1.1 Round 1 (n = 10 per model)

Round 1 used a sample of 10 graded explanations per model. Descriptive statistics are shown in Table 1.

Table 1: Code explanation, Round 1.

Model	Mean	Std. deviation	n
ChatGPT	17.79	0.378	10
Claude	17.38	0.983	10
Gemini	16.34	0.629	10

A one-way ANOVA over the three groups tests the omnibus null:

$$H_0 : \mu_{\text{ChatGPT}} = \mu_{\text{Claude}} = \mu_{\text{Gemini}}$$

$$H_1 : \text{at least one mean differs}$$

Table 2: Code explanation, Round 1: one-way ANOVA table.

Source	SS	df	MS	F	p
Between	11.1740	2	5.5870	11.1383	0.000297
Within	13.5433	27	0.5016	—	—
Total	24.7173	29	—	—	—

The result (Table 2) yields an observed $F(2, 27) = 11.14$ with $p = 2.97 \times 10^{-4}$. The critical value at $\alpha = 0.01$ for the same degrees of freedom is $F_{\text{crit}}(2, 27) = 5.488$. Since the observed $F = 11.14$ exceeds the critical value of 5.488, the omnibus null is rejected at the 99% confidence level: at least one of the three models has a mean rubric grade different from the others.

Having established that at least one model differs, two pairwise one-sided Student’s t -tests were run at $\alpha = 0.01$ to identify where the difference lies.

Best vs. worst: ChatGPT vs. Gemini. The first contrast tested the empirically best-performing model (ChatGPT) against the empirically worst (Gemini):

$$H_0 : \mu_{\text{Gemini}} \geq \mu_{\text{ChatGPT}}$$

$$H_1 : \mu_{\text{ChatGPT}} > \mu_{\text{Gemini}}$$

The test yielded $t_o = 6.2483$ with $df = 9$ and $p = 7.5 \times 10^{-5}$. Since $p < \alpha = 0.01$, H_0 is rejected at the 99% confidence level: ChatGPT is statistically better than Gemini. With this result Gemini is eliminated from the code-explanation competition.

ChatGPT vs. Claude. The second contrast compared the two remaining models:

$$H_0 : \mu_{\text{Claude}} \geq \mu_{\text{ChatGPT}}$$

$$H_1 : \mu_{\text{ChatGPT}} > \mu_{\text{Claude}}$$

The test yielded $t_o = 1.2311$ with $df = 9$ and $p = 0.1247$. Since $p > \alpha = 0.01$, H_0 cannot be rejected: at the 99% confidence level, Round 1 does not allow us to distinguish ChatGPT from Claude.

The two contrasts are summarised in Table 3.

Table 3: Code explanation, Round 1: pairwise one-tailed Student’s t -tests.

Contrast	t_o	df	p	Decision
ChatGPT > Gemini	6.2483	9	0.000075	Reject H_0 at 99% confidence
ChatGPT > Claude	1.2311	9	0.124748	Do not reject H_0 at 99% confidence

The inconclusive ChatGPT vs. Claude result has two plausible causes: (i) the rubric showed a strong ceiling effect at the top end, with both ChatGPT and Claude scoring at or near the maximum on most snippets, and (ii) the sample size $n = 10$ provides limited power for a small effect.

Round 1 therefore eliminates Gemini from the code-explanation comparison but leaves the top two models tied.

5.1.2 Round 2 ($n = 20$, ChatGPT vs. Claude)

To resolve the tie, a second round was executed restricted to ChatGPT and Claude. Two changes were made: the sample size was doubled to $n = 20$ per model, and the rubric was tightened, with partial credit awarded less generously, especially on items (ii) completeness and (iii) clarity. Additionally, responses that exceeded the ten-line limit specified in the prompt were penalised for failing to comply with the formatting constraint, a pattern observed more frequently in Claude’s outputs, which tended to produce significantly longer responses than instructed. The new rubric was fixed before grading began.

Table 4: Code explanation, Round 2.

Model	Mean	Std. deviation	n
ChatGPT	18.16	0.679	20
Claude	16.25	1.509	20

A new ANOVA was run over the two surviving models, with omnibus hypotheses:

$$H_0 : \mu_{\text{ChatGPT}} = \mu_{\text{Claude}}$$

$$H_1 : \mu_{\text{ChatGPT}} \neq \mu_{\text{Claude}}$$

Table 5: Code explanation, Round 2: one-way ANOVA table (two groups).

Source	SS	df	MS	F	p
Between	36.4810	1	36.4810	26.6467	0.000008
Within	52.0243	38	1.3691	—	—
Total	88.5053	39	—	—	—

The result (Table 5) yields an observed $F(1, 38) = 26.65$ with $p = 8 \times 10^{-6}$. The critical value at $\alpha = 0.01$ for the same degrees of freedom is $F_{\text{crit}}(1, 38) = 7.314$. Since the observed $F = 26.65$ exceeds the critical value of 7.314, the omnibus null is rejected at the 99% confidence level: the two models have statistically different mean rubric grades.

The pairwise directional t -test ($\alpha = 0.01$) is set up as before, with ChatGPT as the candidate model:

$$H_0 : \mu_{\text{Claude}} \geq \mu_{\text{ChatGPT}}$$

$$H_1 : \mu_{\text{ChatGPT}} > \mu_{\text{Claude}}$$

The test is now decisive:

Table 6: Code explanation, Round 2: pairwise one-tailed Student’s t -test.

Contrast	t_o	df	p	Decision
ChatGPT > Claude	5.1620	19	0.000028	Reject H_0 at 99% confidence

With a tighter rubric and a larger sample, ChatGPT is statistically better than Claude at the 99% confidence level. Combining the two rounds, the overall ranking on the code-explanation task is:

$$\text{ChatGPT} \succ \text{Claude} \succ \text{Gemini}$$

Figure 1 summarises the mean rubric grades obtained by each model across both rounds. The improvement in ChatGPT’s score from Round 1 to Round 2, combined with the drop in Claude’s score under the tightened rubric, is what drives the statistically significant separation observed in Table 6. Gemini has no Round 2 bar as it was eliminated after Round 1.

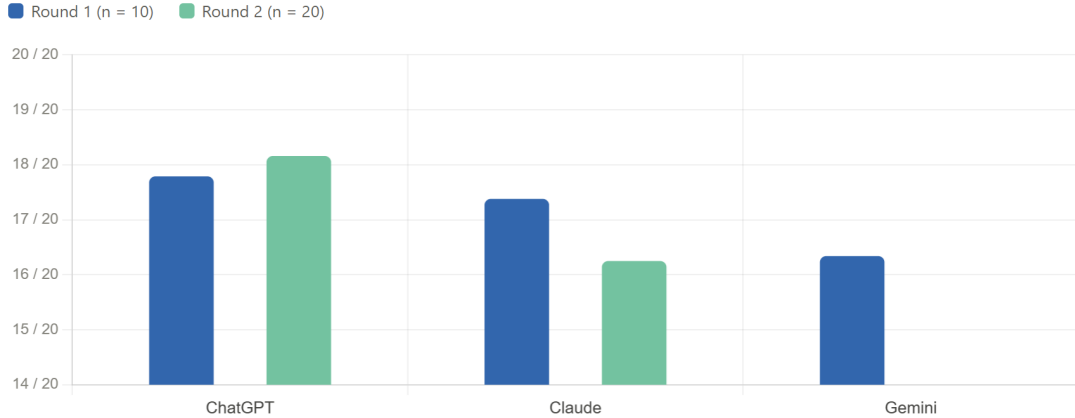


Figure 1: Mean rubric grades per model across Round 1 and Round 2.

It is worth pausing on the methodological lesson here. The same data were *not* re-tested with a different threshold to “manufacture” significance, which would have been a p -hacking pattern. Instead, after a transparent failure to reject, a new round was designed with a different (and stricter) measurement instrument and a larger sample, then run once. This is an acceptable, and indeed common, response to a Round 1 ceiling effect.

5.2 Bug finding

For the bug-finding task, each of the 35 modules was reviewed by each model, giving $35 \times 3 = 105$ observations. Per-model recall and precision averages are reported in Table 7.

Table 7: Bug finding: precision and recall (mean across $n = 35$ cases per model).

Model	Precision	Recall
Gemini	96.19%	84.62%
ChatGPT	90.00%	80.10%
Claude	88.29%	94.57%

A first observation is that the model ordering on precision is exactly the opposite of the ordering on recall. Gemini is the most parsimonious of the three: it reports few bugs and most of what it reports is real, so its precision is high (96%); but it also misses many real bugs, so its recall is the second-highest. Claude shows the opposite behaviour: it is willing to flag aggressively, which costs precision (88%) but rewards recall (95%). ChatGPT sits in the middle on precision and at the bottom on recall.

As argued in Section 2, in a code-review setting the cost of a missed bug is typically larger than the cost of a false alarm: false alarms cost a developer a few minutes of investigation, missed bugs can reach production. Recall is therefore privileged as the primary criterion in the rest of the analysis, and precision is treated as a secondary descriptive metric.

Table 8: Bug finding: recall stratified by difficulty (pooled across models).

Difficulty	n	Mean recall	Std
Easy	24	0.8958	0.1689
Medium	48	0.8698	0.1624
Hard	33	0.8333	0.1349

5.2.1 Step 1: two-way ANOVA on recall

A two-way ANOVA was run on recall with two factors, Model (A) and Difficulty (B), plus their interaction. The three nulls are:

$H_{0,A}$: recall does not depend on the model

$H_{0,B}$: recall does not depend on the difficulty

$H_{0,AB}$: there is no interaction between model and difficulty

The results are shown in Table 9.

Table 9: Bug finding, Step 1: two-way ANOVA on recall, factors Model and Difficulty.

Source	SS	df	MS	F	p
Factor A (Model)	0.3839	2	0.1920	9.3538	0.0002
Factor B (Difficulty)	0.0570	2	0.0285	1.3882	0.2543
Interaction A×B	0.0631	4	0.0158	0.7692	0.5478
Within (error)	2.0316	99	0.0205	—	—
Total	2.5356	104	—	—	—

The interpretation is direct:

- **Model** is highly significant ($p = 0.0002$): the choice of LLM matters.
- **Difficulty** is *not* significant ($p = 0.25$): with the snippet pool used, the planted difficulty level does not, by itself, drive recall down at a detectable rate. This is reassuring, as the models do not collapse on Hard.

- The **interaction** is not significant ($p = 0.55$): the model effect is additive across difficulty levels, so it makes sense to compare the models on overall recall without difficulty-stratified pairwise tests.

5.2.2 Step 2: which models meet the 75% recall floor?

Each model was tested individually against the minimum effectiveness threshold $p_0 = 0.75$. The hypotheses, for each model independently, are:

$$H_0 : p \leq 0.75 \quad \text{vs.} \quad H_1 : p > 0.75 \quad (\alpha = 0.10)$$

The $\alpha = 0.10$ is deliberately looser than the $\alpha = 0.01$ used for code explanation. At this screening stage, the two error types carry asymmetric costs: a *Type I error* (advancing a model that does not truly meet the recall floor) is recoverable, since the model still faces the more demanding head-to-head test in Step 3; a *Type II error* (discarding a model that does in fact meet the floor) is irreversible. A looser α reduces the probability of a Type II error at the cost of a higher, but acceptable, Type I error rate. Step 2 is a *screening* test whose role is only to filter out clearly unfit candidates, not to certify the winner. The test statistic was a one-sided one-proportion z -test:

$$z = \frac{\hat{p} - p_0}{\sqrt{p_0(1 - p_0)/n}}, \quad z_{\text{crit}} = 1.2816.$$

Table 10: Bug finding, Step 2: per-model test $H_0 : p \leq 0.75$ vs. $H_1 : p > 0.75$.

Model	n	$\hat{p}$	z	p -value	Decision
Gemini	35	0.8462	1.3143	0.0944	Reject H_0 at 90% confidence
ChatGPT	35	0.8010	0.6962	0.2432	Do not reject H_0 at 90% confidence
Claude	35	0.9457	2.6740	0.0037	Reject H_0 at 90% confidence

ChatGPT **fails** the screening test. Its observed recall of 80% looks acceptable in absolute terms, but with $n = 35$ and a screening level of $\alpha = 0.10$ it cannot be concluded that its true recall exceeds 75%. By the criterion fixed in advance, ChatGPT is therefore eliminated from the bug-finding competition. This is an important point about the asymmetric stakes: an 80% sample recall is consistent with an underlying true recall that is still below the company’s minimum, and the company should not adopt a tool whose floor it cannot verify.

The 90% confidence intervals for the population recall of each model were computed using the standard confidence interval formula for a population proportion, $\hat{p} \pm z_{0.10/2} \sqrt{\hat{p}(1 - \hat{p})/n}$, yielding: Claude [89.7%, 99.5%], Gemini [76.8%, 92.4%], and ChatGPT [71.5%, 88.8%]. Figure 2 visualises these intervals alongside the 75% recall floor. The interval for ChatGPT is the only one whose lower bound falls below the threshold, which is consistent with the one-proportion z -test result in Table 10 that eliminated it at Step 2.

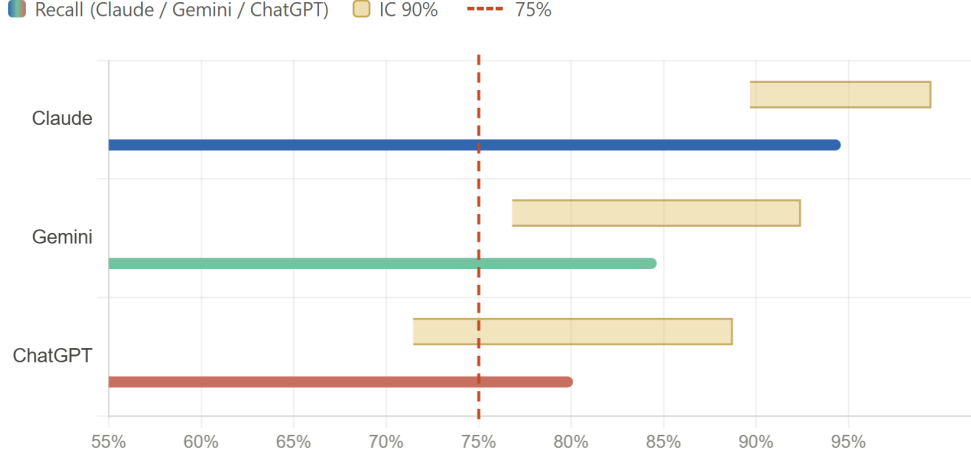


Figure 2: Observed recall (coloured bars) and 90% confidence intervals (shaded bands) for each model. The dashed red line marks the minimum acceptable recall threshold of 75%.

5.2.3 Step 3: head-to-head between the survivors

Among the two surviving models (Claude and Gemini), a one-sided two-proportion z -test was used to test whether Claude’s recall was genuinely better than Gemini’s. With p_1 denoting the empirically higher recall (Claude) and p_2 the lower (Gemini), the hypotheses are:

$$H_0 : p_2 \geq p_1 \quad \text{vs.} \quad H_1 : p_1 > p_2 \quad (\alpha = 0.10).$$

The test statistic was:

$$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}_{\text{pool}}(1 - \hat{p}_{\text{pool}}) \left(\frac{1}{n_1} + \frac{1}{n_2} \right)}}, \quad z_{\text{crit}} = 1.2816.$$

Table 11: Bug finding, Step 3: head-to-head test $H_0 : p_{\text{Gemini}} \geq p_{\text{Claude}}$ vs. $H_1 : p_{\text{Claude}} > p_{\text{Gemini}}$.

Contrast	$\hat{p}_{\text{Claude}}$	$\hat{p}_{\text{Gemini}}$	$\hat{p}_{\text{pool}}$	z	p -value	Decision
Claude vs. Gemini	0.9457	0.8462	0.8960	1.3636	0.0864	Reject H_0 at 90% confidence

The final ranking on recall, restricted to the models that meet the minimum effectiveness threshold, is:

$$\boxed{\text{Claude} \succ \text{Gemini} \quad (\text{ChatGPT eliminated at Step 2})}$$

6 Discussion

6.1 Two tasks, two winners

The result that most deserves emphasis is that **no single model wins both tasks**. ChatGPT is the best at producing natural-language code explanations; Claude is the best at finding bugs in code review. This is the kind of outcome that an organisation deciding on adoption needs to know, because the most common implicit assumption, that one tool will dominate across coding-related tasks, is not supported by the data.

A second-order observation is that the precision/recall trade-off is real and the three models occupy distinct points on it. Gemini behaves like a cautious reviewer (high precision, lower recall); Claude behaves like an aggressive reviewer (lower precision, very high recall); ChatGPT sits in between on precision but is the lowest on recall, which is exactly the wrong corner of the trade-off if recall is the criterion that matters.

6.2 Methodological reflections

The Round 1 / Round 2 structure of the code-explanation experiment deserves a comment. Round 1 did not yield the expected answer (ChatGPT vs. Claude was inconclusive), and the response was not to lower the confidence level or to keep running t -tests until something rejected. Instead, two methodologically defensible steps were taken: a likely cause was identified (a ceiling effect in the rubric), the cause was fixed (a tightened rubric), and the sample size was increased. The new round was then run once, and the result was reported as obtained.

The bug-finding experiment uses a deliberately tiered analysis: an omnibus ANOVA to detect that any difference exists at all, a per-model screening test to eliminate models that cannot demonstrably meet the company’s recall floor, and a head-to-head test among the survivors. The screening step at $\alpha = 0.10$ is meaningfully more lenient than $\alpha = 0.01$ used elsewhere; the asymmetry is intentional and reflects the asymmetry of error costs at that stage of the pipeline (eliminating a model from contention is cheaper than certifying a winner).

6.3 Limitations and Future Work

Despite the care taken in the experimental design, several limitations should be acknowledged.

The snippet pool is relatively small, $n = 10$ per model in Round 1, $n = 20$ in Round 2 for code explanation, and $n = 35$ for bug finding and is also restricted to C and Java. Generalising the conclusions to other languages such as Python or JavaScript, or to large, unfamiliar codebases, would require a separate study with a broader and more diverse set of inputs.

The models themselves are moving targets. A version update between today and three months from now could invalidate the ranking observed here, which means the recommendation to the CTO carries an implicit expiry date and should be revisited periodically.

The code-explanation rubric, even after tightening in Round 2, remains a human construct and carries grader-specific assumptions. A larger and more diverse panel of evaluators, ideally from multiple institutions, would strengthen the reliability of the grades assigned.

Finally, this study does not cover code generation or test case generation, two tasks where the relative ranking of the three models might differ significantly. A natural extension of this work would be to apply the same experimental framework to those tasks, and to combine the results into a broader reliability profile of each tool across the full spectrum of coding-related activities. For the bug-finding task specifically, the effect of programming language was not studied as an independent factor. Future work should investigate whether the observed recall differences between models hold equally for C and Java, or whether certain tools perform better in one language than the other.

7 Conclusions and Recommendation to the CTO

The original research question — *can developers rely on LLM-based tools to support software development activities?* — admits a nuanced “yes, but choose deliberately” as its answer.

Concrete recommendation. If the company can adopt only one tool company-wide, the recommendation depends on which of the two tasks dominates the daily workflow of developers. If code review and bug discovery is the more critical use case, which is the case in regulated or safety-relevant codebases, the recommendation is **Claude**, which is the only model that demonstrably exceeds the 75% recall floor with a comfortable margin (94.57%, $p = 0.0037$) and is also statistically better than the runner-up (Gemini) on recall. If the dominant use case is instead the production of natural-language explanations of code (developer onboarding, documentation, code reading) the recommendation is **ChatGPT**, whose mean rubric grade of 18.16/20 is statistically higher than Claude’s 16.25/20 at the 99% level. Gemini did not win either of the two tasks studied.

Pragmatic recommendation. If a multi-tool licensing strategy is feasible, the right answer is to license both Claude and ChatGPT and to steer developers towards the appropriate tool depending on the task. The marginal cost is small relative to the demonstrated quality difference.

What this study does *not* settle. It does not cover code generation or test case generation, where the conclusions might differ. It does not cover the longer-tail languages or domain-specific code.

8 Contributions of the Group Members

The members of this group worked side by side throughout the semester, with the large majority of the assignment carried out in joint working sessions. Problem framing, hypothesis formulation, snippet selection, ground-truth construction, grading of model responses, statistical analysis, and discussion of results were all done collectively, with all members present and contributing. The decision to run a second round of code explanation, and the design of the tightened rubric used in that round, were taken together after a joint review of the Round 1 results.

For the purpose of acknowledging individual focus areas, the following primary responsibilities can be highlighted, while emphasizing that every member was involved in every part of the work:

- **Daniel Pereira:** primary focus on the ChatGPT model evaluation; responsible for collecting the code snippets used in the code-explanation task, as well as half of the C code snippets used in the bug-finding task; developed the Excel grading template.
- **Eduardo Marques:** primary focus on the Gemini model evaluation; responsible for collecting the remaining half of the C code snippets used in the bug-finding task; developed the scripts for computing mean and standard deviation for both rounds of the code-explanation task, as well as the statistical testing calculator for Round 2.
- **João Cardoso:** primary focus on the Claude model evaluation; responsible for collecting the Java code snippets used in the bug-finding task; developed the grading calculator for Round 1 of the code-explanation task and the statistical testing calculator for the bug-finding task.

All members participated equally in the final review of the report. A detailed contribution history is maintained in the project's GitHub repository (https://github.com/edupmarques/mei_project), alongside all programs, scripts, grading sheets, and other materials used or developed for this work.

9 References

- [1] H. Madeira, *Experimental Methods in Computer Science — Lesson 5: Hypothesis Testing*, lecture slides, Master in Informatics Engineering, DEI-FCTUC, 2025/2026.
- [2] H. Madeira, *Experimental Methods in Computer Science — Lesson 6: Hypothesis Testing — Test for a Proportion*, lecture slides, Master in Informatics Engineering, DEI-FCTUC, 2025/2026.
- [3] H. Madeira, *Experimental Methods in Computer Science — Lessons 8 & 9: Hypothesis Testing — ANalysis Of Variance (ANOVA)*, lecture slides, Master in Informatics Engineering, DEI-FCTUC, 2025/2026.

10 Statement on the use of AI tools

This document was reviewed and refined with the assistance of ChatGPT and similar tools, which helped check grammar, correct typos, and enhance clarity. The charts presented in this report were generated with the assistance of Claude. All ideas, experimental design, data collection, and analysis were carried out exclusively by the authors. The overall content and conclusions remain solely the responsibility of the authors.